

Use Case B - Identifying Similar Historical Claims

Last updated by | Qi, Bill | 20 May 2026 at 17:36 BST

Identifying Similar Historical Claims To Draw Insights From Past Expert Reports

Use Case B creates and processes historical claim rows so they can be reused as similarity candidates for expert-appointment workflows, then exposes export endpoints to inspect the ranked matches and supporting expert documents.

1. Expert Appointment Use Case Overview

1. Use Case Name

Use Case B: identifying similar historical claims to draw insights from past expert reports.

2. Purpose

The implemented flow has two connected goals:

- seed and process historical claims so they become usable similarity candidates
- find and export similar historical claims for a live claim, with a current focus on legal expert appointment use cases

The following image shows the location of the use case in the frontend UI.

Expert appointment

Legal expert needed

YES

- ? Is there ongoing litigation?
- NO Is the insured represented?
- YES Is the reserve recommendation >\$1m?
- NO Is the claims jurisdiction risky?
- YES Did the coverage check fail?
- ? Did the claims of interest check fail?
- ? Did the subrogation check fail?
- YES Did the risk of litigation check fail?
- NO Does a claim with a high similarity score exist in the past?

[View check](#)



Similar historic claims that have received legal advice

A similarity score of 10 represents the highest degree of similarity based on key claim characteristics.

#	Insured name	UCR	Jurisdiction	Loss type	Similarity score	
1	ABCDEFGHIJ	ABCDEFGHIJ	[Jurisdiction]	[Loss type]		9.1
2	ABCDEFGHIJ	ABCDEFGHIJ	[Jurisdiction]	[Loss type]		8.8
3	ABCDEFGHIJ	ABCDEFGHIJ	[Jurisdiction]	[Loss type]		8.7
4	ABCDEFGHIJ	ABCDEFGHIJ	[Jurisdiction]	[Loss type]		8.6
5	ABCDEFGHIJ	ABCDEFGHIJ	[Jurisdiction]	[Loss type]		8.6
6	ABCDEFGHIJ	ABCDEFGHIJ	[Jurisdiction]	[Loss type]		7.0
7	ABCDEFGHIJ	ABCDEFGHIJ	[Jurisdiction]	[Loss type]		3.1
8	ABCDEFGHIJ	ABCDEFGHIJ	[Jurisdiction]	[Loss type]		2.8
9	ABCDEFGHIJ	ABCDEFGHIJ	[Jurisdiction]	[Loss type]		2.8
10	ABCDEFGHIJ	ABCDEFGHIJ	[Jurisdiction]	[Loss type]		2.7

[View all similar claims](#)

3. Implementation Model

The current implementation:

- stores historical similarity candidates directly in `claims.tbl_claims`
- marks those rows with `claim_origin = HISTORICAL`
- manages batch state under `meta.similarity_profile`
- stores historical context under `meta.state.historical_context`
- stores generated embeddings on the claim row via `embedding_text` and `embedding_vector`
- stores ranked live-claim results under `meta.results.similar_claims_ranked`

4. Scope

In scope

- manual or discovery-based seeding of historical claim rows for similarity processing
- discovery of historical claims using Databricks / Insights data with configurable lookback and open-claim thresholds
- Energy-first similarity profiling using the existing ingestion and worker orchestration path
- legal-expert eligibility gating before running the historical similarity profile workflow
- embedding generation and storage on historical claim rows

- similarity candidate retrieval for live claims using historical claim embeddings
- export of top-ranked similar claims and related legal-report documents

5. Assumptions

- there are enough seeded and successfully processed historical claims for a given model to make similarity ranking useful
- historical claims without evidence of legal expert involvement should not proceed through the similarity profile workflow and are marked `SKIPPED`
- the similarity profile is represented by tagged extraction outputs in `meta.conformed_data`, from which the embedding text and vector are derived
- cosine similarity is the first-pass retrieval mechanism; an LLM rerank is used for the live-claim similar-claims output when candidate features are available

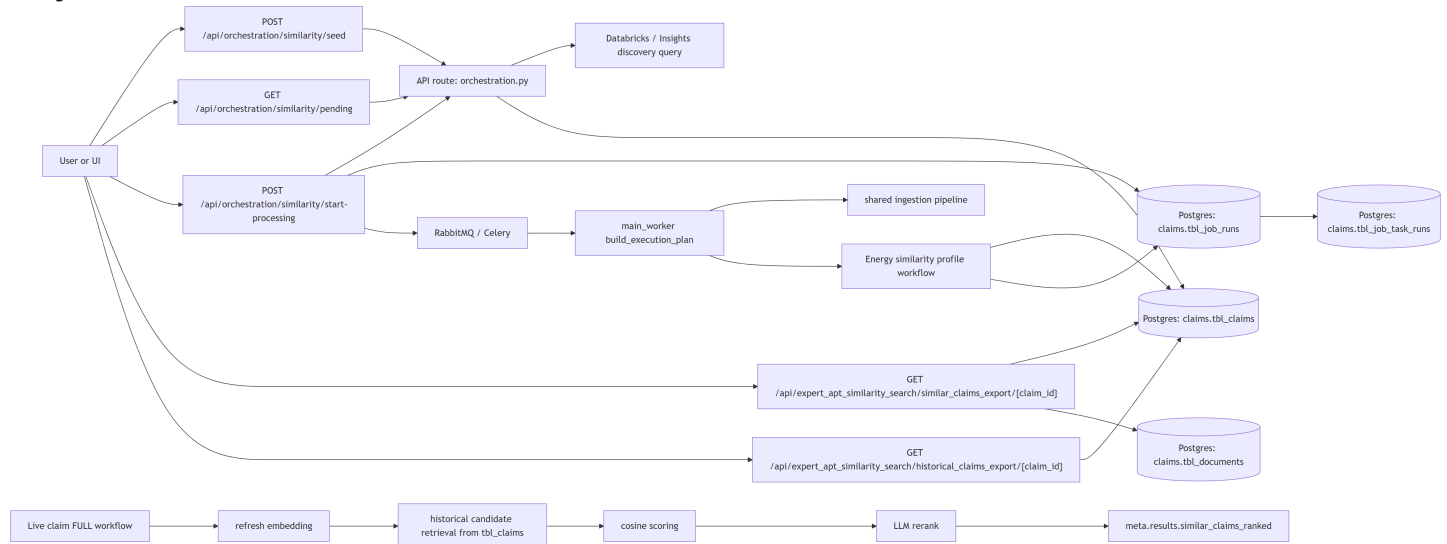
2. User Journey

1. An operator seeds historical claims through `POST /api/orchestration/similarity/seed`.
2. The API either accepts manual records or discovers records from Insights using active model-to-subclass mappings.
3. Seeded rows are inserted into `claims.tbl_claims` as current `HISTORICAL` claims with `meta.similarity_profile.status = PENDING`. Matching current historical rows are backfilled when missing context exists.
4. The operator reviews pending rows through `GET /api/orchestration/similarity/pending`.
5. The operator starts a batch through `POST /api/orchestration/similarity/start-processing`, either by explicit `claim_ids` or by batch filters such as `model_name`, `sub_class_codes`, `lead_types`, and `profile_version`.
6. The API marks selected rows `PROCESSING`, creates `SIMILARITY_PROFILE` jobs, and publishes them to Celery.
7. The main worker reuses the seeded historical claim row, runs the standard ingestion chain plus the Energy similarity-profile workflow, and refreshes the claim embedding.
8. A legal-expert eligibility check runs first. If it fails, the historical claim is marked `SKIPPED` and the remaining similarity-profile workflow is short-circuited.
9. If the workflow completes, Celery signal handlers mark the historical claim `SUCCESS`. If the job fails after selection, it is marked `FAILED`.
10. When a live claim runs through the full expert-appointment flow, the worker builds historical candidates from successful historical claim rows, scores them by cosine similarity, optionally reranks them with an LLM, and stores the output in `meta.results.similar_claims_ranked`.
11. The UI or downstream consumers can export:

- `'GET /api/expert Apt similarity search/similar_claims_export/{claim_id}'` for the top 10 ranked
- `'GET /api/expert Apt similarity search/historical_claims_export/{claim_id}'` for the top 100 h



3. System Architecture



Request flow summary

- **Seed flow:** API route -> optional Insights discovery query -> `tbl_claims` insert or backfill
- **Start-processing flow:** API route -> `submit_celery_task(...)` -> Celery `build_execution_plan` -> ingestion + Energy similarity workflow -> claim row status updates
- **Live similarity flow:** live claim full workflow -> historical candidate retrieval from processed historical claims -> cosine ranking -> LLM reranking -> ranked output persisted to claim meta
- **Export flow:** API route -> read ranked results or historical candidates from `tbl_claims` -> enrich with tagged conformed-data values and legal-report documents -> return response

4. Architecture Mapping

APIs Involved

All routes below are registered under `/api` and protected by the standard app role dependency (`AppUser` , `Admin` , OR `System`).

Endpoint	Method	Purpose	
/api/orchestration/similarity/seed	POST	Seed historical claims for similarity profiling	
/api/orchestration/similarity/pending	GET	List pending historical similarity claims	
/api/orchestration/similarity/start-processing	POST	Mark pending historical rows PROCESSING and queue jobs	
/api/expert Apt Similarity Search/similar_claims_export/{claim_id}	GET	Export the top 10 ranked similar claims for a live claim	
/api/expert Apt Similarity Search/historical_claims_export/{claim_id}	GET	Export top 100 historical cosine matches for a live claim	

5. Backend Codebase

5.1 Repository Structure

The following files are the main implementation surfaces for Use Case B.

- [../src/backend/services/api/app/routes/orchestration.py](#)
Similarity seed, pending, and start-processing API routes.
- [../src/backend/services/api/app/routes/expert Apt Similarity Search.py](#)
Export endpoints for ranked similar claims and raw historical matches.
- [../src/backend/services/api/app/schemas/schemas.py](#)
Request and response schemas for similarity seed, pending, processing, and export payloads.

- [../src/backend/services/api/app/routes/utils/orchestration_util.py](#)
Discovery query orchestration and Celery submission helper.
- [../src/backend/services/api/app/config.py](#)
SQL used for recent-claim discovery and historical similarity seed discovery.
- [../src/backend/services/main_worker/app/tasks.py](#)
`build_execution_plan` and workflow orchestration entry points.
- [../src/backend/services/main_worker/app/managers/claim_manager.py](#)
Claim creation, current-row reuse for historical similarity jobs, and workflow module resolution.
- [../src/backend/services/main_worker/app/managers/execution_manager.py](#)
Includes `SIMILARITY_PROFILE` in both ingestion and task execution plans.
- [../src/backend/services/main_worker/app/workflows/energy/workflow.py](#)
Branches between the standard Energy workflow and the similarity-profile workflow.
- [../src/backend/services/main_worker/app/workflows/energy/task_library/checks.py](#)
Legal-expert eligibility check and gate for historical similarity processing.
- [../src/backend/services/main_worker/app/workflows/energy/task_library/extractions.py](#)
Energy expert-appointment extraction set used to build similarity features.
- [../src/backend/services/main_worker/app/workflows/core/udfs/similar_claims.py](#)
Embedding refresh, historical candidate building, cosine scoring, and LLM rerank logic.
- [../src/backend/libs/common/src/common/embeddings/claim_embeddings.py](#)
Candidate retrieval SQL, embedding helpers, and cosine scoring utilities.
- [../src/backend/libs/common/src/common/celery/logging_signals.py](#)
Job / task logging and final similarity status updates on historical claim rows.
- [../src/backend/libs/common/src/common/database/models.py](#)
Core database models including `tbl_claims`, `tbl_job_runs`, `tbl_job_task_runs`, `tbl_models`, and `tbl_documents`.
- [../src/backend/ops/migrations/alembic/versions/85df67020dbd_job_name_similarity_profile.py](#)
Migration that adds `SIMILARITY_PROFILE` to the `claims.job_name` enum.

5.2 Pseudocode For Business Logic

A. Historical seeding and processing



```

seed_similarity_records(payload):
    records = payload.records
    if records are empty:
        records = discover_similarity_seed_records(
            lob, lookback_years, open_claim_months, profile_version
        )

    load model_id lookup from tbl_models
    load existing current historical tbl_claims rows keyed by (ucr, umr, model_id)

    for each discovered or manual record:
        if current historical row already exists:
            backfill raw_structured_data.sub_class_code when missing
            backfill meta.state.historical_context fields when missing
            continue

        create new current tbl_claims row with:
            claim_origin = HISTORICAL
            raw_structured_data.sub_class_code
            meta.similarity_profile = {
                status: PENDING,
                profile_version,
                updated_at
            }
            meta.state.historical_context = {
                claim_status,
                claim_opened_at,
                claim_closed_at,
                lead_type
            }

get_similarity_pending(limit, model_name, profile_version):
    return current HISTORICAL tbl_claims rows
    where meta.similarity_profile.status = PENDING
    and profile_version matches

start_similarity_processing(payload):
    select current HISTORICAL tbl_claims rows where status = PENDING
    apply optional filters:
        claim_ids
        model_name
        sub_class_codes
        lead_types
        profile_version
        batch_size

    if no rows selected:
        return success response with zero queued

    mark selected rows PROCESSING
    build JobRunRequests with:
        job_name = SIMILARITY_PROFILE
        claim_id = existing historical claim_id
        umr, ucr, claim_model, force_run

    submit_celery_task(...)
    assign returned job_id values back onto tbl_claims rows

```

B. Worker execution for historical similarity jobs

```

build_execution_plan(payload):
    job_name = SIMILARITY_PROFILE
    claim_origin = HISTORICAL
    claim_id = create_claim(... reuse_existing_current_claim=True)
    load claim structured data
    workflow = energy.generate_workflow(job_name=SIMILARITY_PROFILE)
    execution_manager includes both ingestion and task chains
    publish orchestrator to Celery

```

Energy similarity-profile workflow:

1. run legal-expert involvement check
2. if check fails:
 - set meta.similarity_profile.status = SKIPPED
 - short-circuit remaining workflow groups
3. else run expert-appointment extraction subset
4. refresh embedding_vector from embedding_text

Celery terminal updates:

```

SUCCESS -> update current historical claim meta.similarity_profile.status = SUCCESS
FAILURE -> update current historical claim meta.similarity_profile.status = FAILED
SKIPPED -> preserve SKIPPED terminal state on clean exit

```

C. Live similar-claim generation and export

```

live claim similar-claims pipeline:
    refresh live claim embedding_vector
    fetch current historical tbl_claims rows where:
        claim_origin = HISTORICAL
        embedding_vector is present
        similarity_profile.status not in (PENDING, PROCESSING, FAILED, SKIPPED)
        model matches live claim

```

```

    optionally prefilter by tagged expert-appointment pre-filters
    compute cosine similarity
    keep top K candidates
    rerank with LLM using query claim features + candidate embedding text
    if rerank fails, fall back to cosine order
    persist final list to meta.results.similar_claims_ranked

```

```

similar_claims_export(claim_id):
    read live claim meta.results.similar_claims_ranked
    keep top 10 by llm_rank
    fetch each similar claim meta and legal report documents
    return tagged pre-filter, embedding-feature, and detail values

```

```

historical_claims_export(claim_id):
    parse live claim embedding_vector
    fetch historical candidates directly from tbl_claims
    cosine-rank top 100
    return tagged values without document enrichment

```

5.3 Event Triggering

- Use Case B is currently **manually triggered** from the API. There is no implemented scheduler for automatic reseeding or reprocessing.
- POST /api/orchestration/similarity/start-processing is the main event that publishes similarity-profile jobs.
- submit_celery_task(...) creates tbl_job_runs records and publishes build_execution_plan tasks to Celery.
- main.workflow_orchestration executes task groups in sequence, with each group able to run tasks in parallel.
- Celery signal handlers in logging_signals.py update both job/task tracking tables and the claim-native similarity status stored on the current historical claim row.

5.4 Data Impact

Tables Read

Application database

- `claims.tbl_claims`
Read during seeding, pending selection, processing selection, candidate retrieval, export, and claim reuse.
- `claims.tbl_models`
Read to resolve model IDs and model names.
- `claims.tbl_model_mapping`
Read to determine active subclass-code mappings used in historical discovery.
- `claims.tbl_job_runs`
Read for in-progress checks and job status handling.
- `claims.tbl_job_task_runs`
Read and updated for Celery task observability.
- `claims.tbl_documents`
Read by the export flow to retrieve expert documents for similar claims.

External data sources

- Databricks / Insights warehouse tables referenced by `INSIGHTS_SIMILARITY_DISCOVERY_SQL_QUERY`
Used to discover eligible historical claims, claim lifecycle, lead type, and subclass information.
- OpenText / document retrieval sources used by the standard ingestion flow
Used indirectly when similarity-profile jobs run the shared ingestion pipeline.

Tables Written

- `claims.tbl_claims`
New historical rows are seeded here; existing rows are backfilled here; batch status, `job_id`, embeddings, and live ranked results are all persisted here.
- `claims.tbl_job_runs`
One row per queued similarity-profile job.
- `claims.tbl_job_task_runs`
Task-level execution tracking for worker tasks.

Important implementation note

The current use-case implementation keeps both historical similarity profiles and live similarity results on `tbl_claims`.

5.5 Dependencies

Internal Dependencies

- API routing and schema layer in the `services/api` package
- shared Celery publication helper `submit_celery_task(...)`
- shared claim lifecycle management in `ClaimManager`
- shared orchestration planning in `ExecutionManager`
- Energy workflow definitions and expert-appointment extraction/check libraries

- shared similarity helpers in `core/udfs/similar_claims.py`
- shared embedding utilities in `common.embeddings.claim_embeddings`
- shared Celery logging / status signaling in `common.celery.logging_signals`

External Dependencies

- PostgreSQL with JSONB and vector storage for claim state and embeddings
- RabbitMQ / Celery for asynchronous orchestration
- Databricks / Insights for historical seed discovery and structured claim data
- OpenText / document retrieval stack used by the ingestion pipeline
- LLM and embedding providers used through the `aiutil` abstraction for:
 - legal-expert eligibility checks
 - expert-appointment extraction tasks
 - claim embedding generation
 - similar-claim reranking

6. Summary Of Implemented Behavior

Use Case B is currently implemented as a claim-native historical profiling and live retrieval flow. Historical candidates are seeded into `tbl_claims`, processed through a dedicated `SIMILARITY_PROFILE` worker path, filtered by legal-expert eligibility, and stored with embeddings on the same claim row. Live claims then retrieve those successful historical rows, score and rerank them, and expose the result through export endpoints tailored to expert-appointment use cases.